OWNER REFERENCE

How Star Player of the Week is built

A living document — updated as the app changes.

A plain reference for talking about this app with full confidence — what it's built with, how the data is organized, what's protected against, and how a change actually goes live.

01 WHAT IT'S BUILT WITH

FRONTEND

React + TypeScript

Built with Vite. Styled with Tailwind CSS on a small custom set of colors, spacing, and type sizes.

BACKEND

Firebase

No custom server. Firebase handles sign-in (email + password) and the database.

DATABASE

Cloud Firestore

Data is stored as documents in named groups called "collections," not tables. The app gets live updates automatically, without asking for them.

HOSTING

Firebase Hosting

The site is hosted on Google's servers and deployed from the command line. There's no separate server to maintain.

UI PARTS

Radix UI + custom

Radix supplies accessible dialogs and toasts. Everything else — buttons, fields, badges — is built by hand to match the brand.

TESTING

Vitest + a Firestore test copy

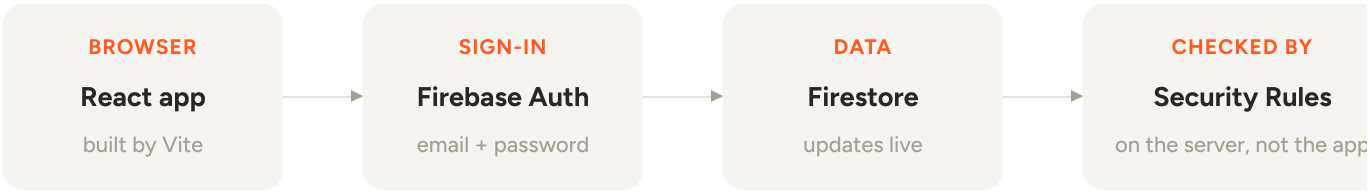
Unit tests check plain logic. A separate test suite runs the real security rules against a local test copy of the database, to prove access rules actually work.

CODE STORAGE

GitHub

One main branch. Every change is saved as a commit before it goes live.

02 HOW A REQUEST FLOWS THROUGH IT



There's no backend server deciding permissions — the app talks to the database directly, and a separate file called **Security Rules** decides what any signed-in person is actually allowed to do. That matters: nobody can get more access just by changing what the app's own code does. Only changing the rules themselves changes what's allowed.

03 HOW THE DATA IS ORGANIZED

COLLECTION	WHAT IT HOLDS	WHO CAN READ IT
`sotw_profiles`	Name, email, and role for every teammate	**Everyone signed in**
`sotw_balances`	Each person's Blacfox Dollar balance	**That person, admin, finance**
`sotw_myvote`	Each person's own current vote	**Only that person**
`sotw_tally`	Anonymous vote counts per person	**Admin, after reveal only**
`sotw_meta/settings`	Is voting open, has it been revealed, who's finance, the winner	**Everyone signed in**
`sotw_payout_requests`	Pending, approved, and rejected cash-out requests	**Requester, admin, finance**
`sotw_payouts`	A permanent record of every bonus actually paid	**Admin, finance**
`sotw_balance_adjustments`	A record of manual balance fixes — who changed what, and from what to what	**Admin, finance**

Every name starts with `sotw_` — a simple prefix so this data can share a database with something else later without clashing.

04 WHAT'S PROTECTED

- ✓ **Votes stay private.** Nobody — not even an admin — can see who voted for whom. Only anonymous totals exist, and even those are hidden until the week is revealed.
- ✓ **Money can't be created out of nowhere.** A finance holder can only lower a balance, never raise one — that's enforced by the database itself, not just the screen the app shows.
- ✓ **The weekly bonus is paid exactly once.** Revealing the winner and paying them happen together, as one step — a dropped connection can't leave someone revealed but unpaid, or paid twice.
- ✓ **Every manual money change leaves a record.** Fixing a balance by hand always writes down who changed it and what it was before.

05 SHIPPING A CHANGE

Two separate steps, kept apart on purpose:

```
# app code — what people see and use
firebase deploy --only hosting

# security rules — who's allowed to do what
firebase deploy --only firestore:rules
```

Keeping these separate means a normal update to the app can never accidentally change who's allowed to do what. That only ever happens on purpose, as its own step.

06 WHAT'S HAPPENED SO FAR

- **Rebuilt from one big file**

  The app used to be a single large HTML file. It's now a proper React + TypeScript codebase, easier to maintain and grow. The old version is being phased out.

- **Sign-in screen redesigned**

  Went through a few passes to land on a clean, centered design.

- **Security check-up**

  A full review found and fixed real gaps — including a way anyone signed in could inflate vote counts, and a way the finance role could raise balances instead of only lowering them.

Made the admin tools self-serve

Adding people, fixing a balance, and recovering from a stuck state can now all be done from the app — no direct database edit needed.
